

BizQuery Project Proposal

By: Ben Hug (benhug2), Danny Guller (dguller2),
Brandon Lee (bylee3), Viswanath Vadlamani (sv56)

GitHub Repository

<https://github.com/benhug2/fa25-cs411-team007-BizQuery>

Project Title

BizQuery - Personal Finance Tracker

Project Summary

BizQuery is a web-based personal finance tracker that allows users to record expenses and income, categorize transactions, set budgets, and visualize trends. The system will use an SQL database to store user accounts, transaction history, categories, and budgeting data. The goal of the project is to provide users with clear insight into their spending habits. This will allow users to make informed decisions through visualizations and budget monitoring.

Description

Many people struggle tracking their daily spending, leading to lack of financial awareness and poor organization. This lack of organization can lead to overspending, missed opportunities to save money, and limited awareness of where money is actually going. Existing financial tools can be complex, expensive, or not customizable. This leaves many users without an effective way to manage personal finances.

Our application addresses this problem by offering a user-friendly platform where users can log financial transactions, categorize expenses, and set budget limits. The system will be powered by a relational database, ensuring the data is stored efficiently, securely, and supporting multiple users. SQL queries will drive core functionality, enabling users to visualize financial trends, and receive real-time data.

Creative Component / Technically Challenging Features

One creative component inside of the project is a data visualization dashboard. This would have dynamic charts and graphs to show expenses, spending, and many more data points. The dashboard would allow the user to drill-down into a specific category inside the chart to see a detailed breakdown of that data. This would be implemented using a visualization library such as Chart.js or matplotlib.

Other creative components of the project would be automated budget alerts and API integrations. The budget alerts would notify the user when they are nearing or exceeding a budget limit. This would be implemented using SQL triggers or checks that are scheduled. The API integrations would import bank transaction information data from open banking APIs. This would allow the user to view real-time banking information.

Usefulness

The project helps users develop financial discipline by tracking spending and savings. The basic functions of this application include:

- Adding, updating, deleting, and viewing transactions
- Generate financial reports after each month
- Set budget limits per category or overall
- Filter and search transactions by date, category, or amount

Some more complex features include the visualization dashboard with real-time updates and possibly budget alerts. Similar applications include: Mint, You Need A Budget, and Personal Capital. The difference between our application and these is our finance tracker is a custom application focused on database design. It is open-source and adaptable to certain requirements, which is unlike commercial applications.

Realness

One of our data sources is over a year's worth of real spending data. It is in an Excel (.xlsx) format and contains fields such as Date, Amount, Payment Method, and Category. The data is roughly 18 months of data with several hundred transactions. This provides realistic data for testing database queries, ensuring that features such as categorization and budget tracking are performing correctly.

The second form of data will be using user-generated data. The SQL tables will be populated with realistic test data, which includes transactions, budgets, and users. There will be several hundred transactions with sample rows for testing scalability. The supplemental data allows us to scale beyond the individual records, ensuring the system supports multiple users and high transaction volumes.

Functionality

The user can create, read, update, delete, and filter using our application.

Create: The user will be able to register an account, add transitions, and set budgets.

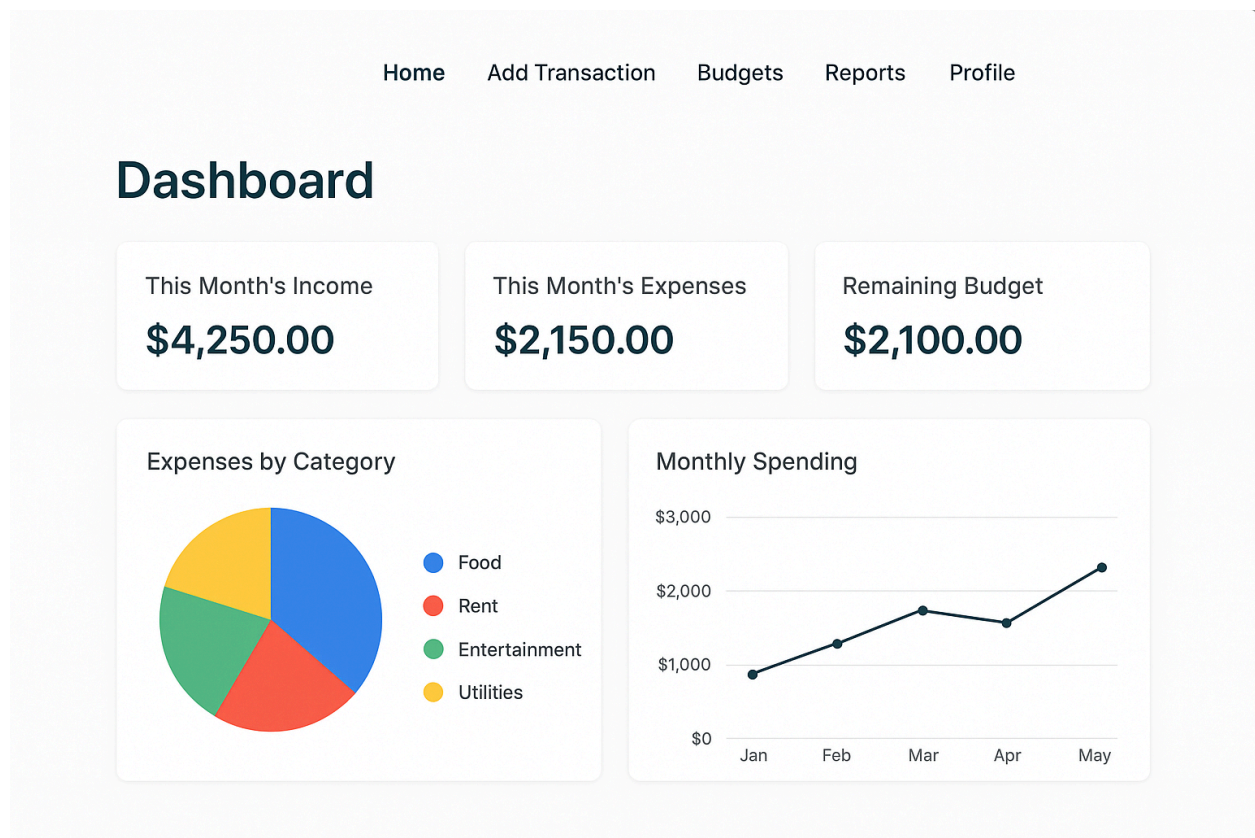
Read: The user will be able to view transaction history, budgets, and summary reports.

Update: The user will be able to edit transactions, update budgets, and change categories.

Delete: The user will be able to remove transactions or budgets.

Filter: The user will be able to query transactions by date range, category, or keyword.

Low-Fidelity UI Mockup



Here is an example of what our UI might look like. It features the user's income, spending, expenses, and budget. It also shows the user's expenses by category and their monthly spending as visualizations. There will be other tabs where the user can look at additional information and add data to their profile.

Project Work Distribution

For the distribution of work, we will assign one member of the team to one of the following categories: Database Design & SQL, Backend Development, Frontend Development & UI/UX, and Data Visualizations & Integration. First, Ben will be responsible for Database Design & SQL. He will be in charge of designing the relational database, writing SQL scripts for table creation and queries, and ensuring normalization for data integrity. Second, Danny will be in charge of Backend Development. He will be

in charge of connecting the frontend with the SQL database, implementing operations (CRUD) for transactions, budgets, and users, and handling authentication of the user. Next, Viswanath will be responsible for Frontend Development & UI/UX. He will be responsible for developing the user interface, implementing mockups into HTML/CSS/JS, and integration with the backend. Finally, Brandon will be responsible for the Data Visualizations & Integration. He will be in charge of integrating visualization libraries (Chart.js, matplotlib), preparing the datasets, and possibly implementing budget alerts and summary reports.